THRASHGUARD

Deep-Learning-Based Thrashing Detection

Invention / Project Technical Report

Prepared by
Adish Kumar Gupta | Lakshay Sharma

Operating Systems • Time-Series Deep Learning • PatchTST • Flask Dashboard

Based on the current project implementation, training output, presentation, and selected literature

Report scope and implementation status

Important status note. This report outlines the details of the current ThrashGuard prototype. In the given training process, an 800-row dataset was utilized and the best observed test accuracy achieved was 75.17% in training and a final accuracy value was 68.97%. At present, the browser version uses dummy metric values and the end-to-end data gathering from the live Ubuntu machine to the dashboard is to be integrated in the future. Thus, the results presented can be regarded as results from the prototype stage.

The term “invention” appears in accordance with the required report template. This report does not legally assert anything about the patentability of the invention or about its novelty based on prior art search.

1. Title of the invention

ThrashGuard: A PatchTST-Based System for Memory Pressure and Thrashing Classification

2. Field / Area of invention

ThrashGuard operates in the domains of operating systems, virtual memory, system performance monitoring, time series machine learning, deep learning, and software observability. The algorithm analyzes metrics from an operating system’s resources over time to classify conditions of the system regarding memory and give early warning about the possible development of thrashing.

- The main domain: OS memory performance monitoring.
- The technology: multivariate time series classification via a PatchTST-like Transformer model.
- The implementation: Python, PyTorch, Hugging Face Transformers, scikit-learn, Flask, and HTML-based dashboard.
- The target environment: Linux/Ubuntu (currently the browser demo uses dummy data).

3. Literature Survey

The literature surveyed includes the operating system principles of thrashing, pressure measurements, deep learning based predictions of page faults, Transformers architecture and time series modeling using patches. All these contribute to the design, but no single literature describes the entire ThrashGuard design on its own.

Reference	Contribution	Relevance	Limitation for this project
Denning (1968) [1]	Explains thrashing, working sets, and memory demand relative to available RAM.	OS foundation; motivates detecting pressure before performance collapses.	Classical analysis; no learned multi-metric classifier.
Linux PSI documentation [2]	Describes Pressure Stall Information for CPU, memory, and I/O stalls.	Context for OS pressure monitoring.	Monitoring interface, not a four-class learned predictor.
Vaswani et al. (2017) [3]	Introduces Transformer and self-attention.	General sequence-modelling foundation.	Not designed specifically for OS thrashing.
Nie et al. (2023), PatchTST [4]	Segments time series into patches for Transformer-based forecasting and representation learning.	Informs the patch-based architecture adapted here.	Original work focuses on forecasting/representation learning, not this four-class task.
Chuah et al. (2025) [5]	Evaluates RNN, LSTM, and TCN for predicting major page faults using cluster logs and resource data.	Supports applying deep learning to memory-related system events.	Predicts page-fault events in clusters, not the same thrashing stages.
Al Maruf & Chowdhury (2020) [6]	Leap uses access-pattern-based prefetching to improve remote-memory access latency.	Supporting context on memory-access behaviour and performance costs.	Optimizes remote-memory access; does not classify thrashing.
Silberschatz et al. [7]	Textbook treatment of OS memory management and virtual memory.	General OS background.	Not a learned early-warning method.

Conclusion: The above studies serve as inspiration for looking at thrashing as an operating system performance issue, resource metrics analysis over time, and Transformer modeling using patches. These concepts have been brought together in ThrashGuard.

4. Summary and background of the invention (Address the gap / Novelty)

Abstract

ThrashGuard is a prototypical system designed for operating system condition classification that is related to memory pressure and thrashing. The system makes use of five features namely page-faults, swap I/Os, free memory, CPU utilization, and run-queue length in a multivariate time series sequence containing 16 timesteps. The features are normalized using StandardScaler while a PatchTST Transformer is used as the classifier with four labels namely Normal, Memory Pressure, Approaching Thrashing, or Thrashing. Flask backend is used to load the model and scaler and provides endpoints related to prediction on the dashboard. Currently, the training phase uses a dataset consisting of 800 rows with 200 samples per class, the best observed accuracy for test set in training phase is 75.17% while the final accuracy of evaluation phase is 68.97%.

Introduction

The technique of virtual memory is used by operating systems to allow programs to run even if their entire working sets cannot be kept in physical memory at once. Whenever there is a shortage of physical memory space, pages might shift between physical memory and slower secondary storage. But if the computer takes too much time to perform shifting of pages rather than performing any productive process on behalf of applications, then this might lead to extremely poor performance, which is called thrashing. Single metrics like page fault rate are insufficient to detect thrashing because of memory pressure.

Background

A page fault happens when a process accesses a page that is not currently present in the physical memory. All page faults do not necessarily require disk I/O operations; major page faults are likely to include I/O and are usually costly. Swap I/O refers to page transfers to or from swap space, whereas Free memory gives an estimate of the free physical memory. CPU usage and Run queue length provide additional insights into the level of activity in the system. Taken collectively and analyzed over time, these metrics might indicate memory pressure.

The proposed method currently has an input window of 16 × 5. Using a patch length of 4 and a stride of 4 results in four patches. These patches are then fed into a Transformer encoder with pooling and classification head on top. The backend uses the pretrained model and scaler.

Research Gap

• One metric ambiguity: a high page-fault rate can happen due to reasons different from thrashing; a multi-metric perspective can help here.

• Too narrow a time scope of simple threshold-based algorithms: static thresholds usually refer to the current situation and do not cover the trends of several metrics.

• Special task gap addressed here: while PatchTST solutions deal with forecasting/representation learning, the page-fault prediction approach predicts specific page-fault events. The current project studies adapting PatchTST to four-class classification of OS states.

Novelty claim. At the project level, novelty lies in the implemented combination of five OS metrics, input data size of 16 steps, PatchTST-based four-class classification, prediction service with Flask framework, and the dashboard. The novelty claim does not mean that the research result is global novel or can be patented; this should be checked via prior art search.

5. Objective(s) of invention

1. Utilize more than one OS metric to represent system behavior with respect to memory.
2. Capture a multivariate time-series input of fixed length that comprises 16 instances of five metrics.
3. Develop a model similar to PatchTST, which can classify into Normal, Memory Pressure, Approaching Thrashing, and Thrashing categories.
4. Create a model that enables inference by utilizing pre-trained model and fitted scaler.
5. Create a flask-based backend for the model to expose predictions to the user through the browser.
6. Establish a baseline model and identify validation required before any live system testing.

6. Working principle of the invention (in detail)

Step 1 – Metric inputs. Five metrics are considered: page fault rate, swap io, free memory mb, CPU usage, and queue length. These would be gathered in a deployed system from Linux, but the demo uses made-up numbers for the browser interface.

Step 2 – Data validation. The training algorithm examines the data shape, missing values, class balance, and simple errors. The provided dataset has 800 observations, 8 columns, 0 missing values, and four classes of 200 observations each.

Step 3 – Feature scaling. The StandardScaler is applied to the training dataset to scale the five numerical features. The fitted scaler is kept for inference use to perform the same scaling on the inputs.

Step 4 – Sequencing. Features are organized into 16 timesteps to create a 16 x 5 input. The generated number of training sequences and test sequences in the training process is 625 and 145 respectively.

Step 5 – Patching. A patch size of 4 and a stride of 4 divide the 16 timesteps into four consecutive patches.

Step 6 – Transformer encoding. The patches are embedded and encoded using the Transformer encoder. The configured model consists of 2 encoder layers with model dimension 64, attention head size 4, and feed forward size 128

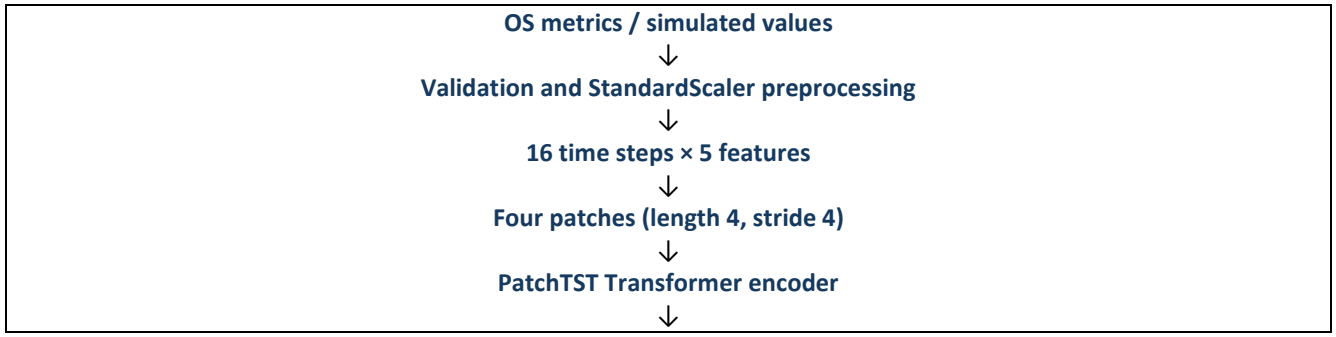
Step 7 – Classification. Pooled sequence is sent to classification head where class probabilities are computed for choosing one of four labels corresponding to system states.

Step 8 – Training and saving. Cross-entropy loss function evaluates classification accuracy and AdamW optimizes weights. Config file provides the following parameters: learning rate 0.0005, batch size 32 and 30 epochs. Model and scaler checkpoints are saved after training.

Step 9 – Inference. On start, Flask initializes the saved model and scaler. Needed input sequence is prepared, scaled, prediction is made and returned via API.

Step 10 – Display on dashboard. Browser side displays collected metrics and predicted system state. Current implementation relies on simulated data. Ubuntu-based collector-to-dashboard connection is incomplete.

End-to-end flow



Pooling + classification head
↓
Four-class prediction and probabilities
↓
Flask API → browser dashboard

7. Description of the invention in detail

7.1 Input metrics

Feature	Meaning	Role in interpretation
Page-fault rate	Rate of page-fault events observed over an interval.	May rise when pages are not resident in RAM; fault type and workload matter.
Swap I/O	Amount/rate of swap activity over an interval.	Sustained high activity can indicate paging pressure.
Free memory (MB)	Estimate of available physical memory.	A falling trend can indicate reduced memory headroom.
CPU usage (%)	CPU utilization indicator.	Adds activity context; high or low values alone do not prove thrashing.
Run-queue length	Runnable tasks waiting for CPU service, as represented by the collector.	Helps indicate contention or system load.

7.2 Model configuration

Parameter	Current implementation
Input window	16 time steps × 5 features
Patch length / stride	4 / 4; four patches per window
Model family	PatchTSTForClassification / PatchTST-based Transformer classifier
Transformer layers	2
Model dimension	64
Attention heads	4
Feed-forward dimension	128
Loss / optimizer	Cross-entropy / AdamW
Learning rate	0.0005
Batch size / epochs	32 / 30
Classes	Normal; Memory Pressure; Approaching Thrashing; Thrashing

7.3 Software architecture

- preprocessing.py: validates/prepare the data, feature scaling, and builds time series windows.
- train.py: trains the classifier, validates checkpoints, and saves the model and scaler.
- evaluate.py: evaluates the model pipeline and provides classification metrics.
- app.py: loads the trained model and scaler, provides Flask routes, and performs inference.
- templates/index.html: displays the web interface; the demo currently generates mock metric data.

Artifacts that have been saved. The training process has saved the model to models/patchtst, and the scaler is saved to models/scaler.pkl. The logs from successful local startup indicate that both model and scaler were loaded.

7.4 Output states

Class	Interpretation
0 — Normal	Indicators resemble the normal class in the training data.
1 — Memory Pressure	Indicators resemble a memory-stressed state.
2 — Approaching Thrashing	Indicators resemble the intermediate high-pressure class intended

	to provide an early-warning stage.
3 — Thrashing	Indicators resemble the severe paging/performance-degradation class.

A Recovery state is presented as a futuristic concept within the discussion; however, the existing trained model has four categories that do not include Recovery.

8. Experimental Validation Results

8.1 Dataset and setup

- Size of data: 800 rows.
- Balance of classes: 200 rows each of Normal, Memory Pressure, Approaching Thrashing, and Thrashing.
- Check claims: absence of null values; absence of negative page-fault, swap, memory, and run-queue values; CPU values between 0-100.
- Training script splitting: 640 raw rows for training and 160 rows for testing; 625 training sequences and 145 test sequences after sequence formation.
- Local environment: Python 3.12 virtual environment, PyTorch CPU version, Flask, NumPy, Pandas, scikit-learn, and Transformers.

Validation disclaimer: The provided dataset does not have independent recordings in chronological order of real-machine traces. The results should be considered as prototype results based on this dataset and not as a generalization on an arbitrary Linux workload.

8.2 Training results

Measure	Observed result
Best test accuracy observed during training	75.17% (epoch 12)
Final reported evaluation accuracy	68.97%
Final training accuracy	82.40% (epoch 30)
Training loss at epoch 30	0.4472
Training schedule	30 epochs; CPU execution

8.3 Per-class results from final evaluation

Class	Precision	Recall	F1-score	Support
Normal	0.80	0.80	0.80	35
Memory Pressure	0.64	0.64	0.64	36
Approaching Thrashing	0.55	0.58	0.57	36
Thrashing	0.78	0.74	0.76	38
Macro average	0.69	0.69	0.69	145
Weighted average	0.69	0.69	0.69	145

8.4 Confusion matrix

Rows are actual classes and columns are predicted classes, ordered Normal, Memory Pressure, Approaching Thrashing, Thrashing.

Actual \ Predicted	Normal	Memory Pressure	Approaching Thrashing	Thrashing
Actual Normal	28	5	2	0
Actual Memory Pressure	7	23	5	1
Actual Approaching Thrashing	0	8	21	7

Actual Thrashing	0	0	10	28

8.5 Interpretation

However, the final evaluation was able to recognize 100 out of 145 test sequences, amounting to an accuracy of 68.97%. The F1-score for Normal is 0.80, whereas Approaching Thrashing has an F1-score of 0.57 and is the hardest class in this run. From the confusion matrix, we can see that there are some Memory Pressure instances which were recognized as Normal and also some of the adjacent states with high memory pressure.

These two accuracies are different from those obtained in the provided run. An evaluation should use an unused test set which is not referred to during the training or model selection process.

8.6 Limitations and further validation required

- The current browser dashboard employs artificial data. The end-to-end Ubuntu live metric stream remains to be validated.
- The number of balanced rows in the dataset is only 800 which is inadequate for establishing generalization across different machines, workloads, and memory configurations.
- The procedure for creating and splitting temporal windows should be looked at in order to avoid overlapping and similar windows leaking between training and testing datasets. Chronological splitting based on trace/session would be more appropriate.
- Test PatchTST against some baselines such as thresholds, logistic regression, random forest, and simple sequence models.
- Provide repeated-run variability, class-specific metrics, calibration, false alarm rate, and missed warning rate. Accuracy is not enough to demonstrate early warning.
- The Recovery class is missing in the current four-class model.
- The current prototype does not terminate any process or adjust resource allocation by OS.

9. References

1. [1] P. J. Denning, “Thrashing: Its Causes and Prevention,” Proceedings of the AFIPS Fall Joint Computer Conference, vol. 33, 1968, pp. 915–922.
2. [2] J. Weiner, “PSI — Pressure Stall Information,” Linux Kernel Documentation. Available: https://docs.kernel.org/6.9/accounting/psi.html
3. [3] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
4. [4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers,” International Conference on Learning Representations (ICLR), 2023. arXiv:2211.14730.
5. [5] E. Chuah, A. Jhumka, S. Narasimhamurthy, and A. Ayesh, “Deep Learning-Based Prediction of Major Page Faults in Cluster Systems,” CCF Transactions on High Performance Computing, vol. 7, pp. 413–430, 2025. doi: 10.1007/s42514-025-00233-2.
6. [6] H. Al Maruf and M. Chowdhury, “Effectively Prefetching Remote Memory with Leap,” Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC ’20), 2020.
7. [7] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.
8. [8] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” International Conference on Learning Representations (ICLR), 2019.

End of report. Update this document as live-data integration and further